Graphs + Matrices

When to use)

- searching graphs or matrices
- DFS : explore all possible paths
- BFS : find the shortest path
- Topological Sort: order tasks based on dependencies

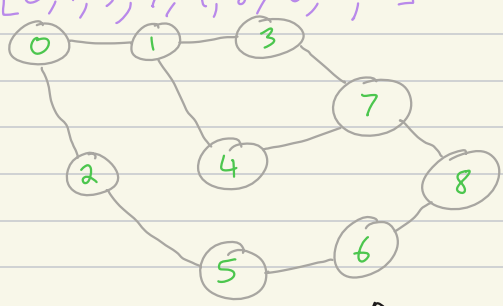
What is it)

- DFS traverses <u>as deep as possible</u> along each branch before backtracking, prioritizing visiting nodes in a recursive or stack-based manner. BFS explores all neighbors of a node <u>before moving deeper</u>, traversing by use of queue. For DFS use recursion and a visited set to keep track of nodes, while BFS uses iteration with a queue <u>and</u> a visited set to track.

- In a matrix neighbors are up/down/left/right cells, while a graphs neighbors are found in the adjacent list.

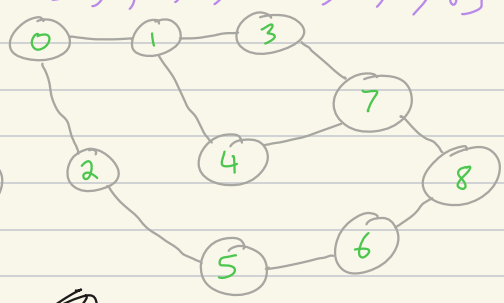
Example Diagrams)

[0, 1, 3, 7, 4, 8, 6, 5, 2]



DFS

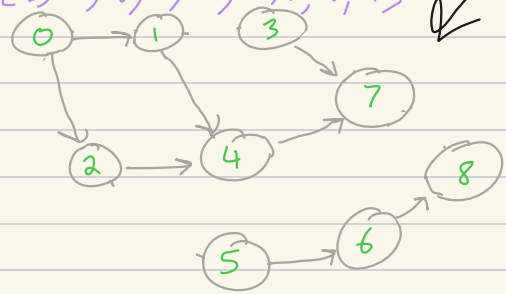
[0, 1, 2, 3, 4, 5, 6, 7, 8]



BFS

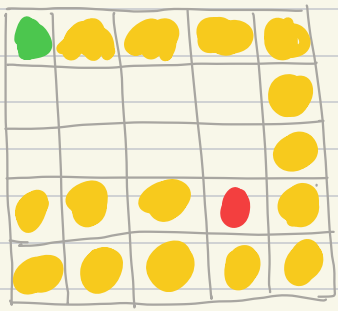
graph

[5, 6, 8, 3, 0, 2, 1, 4, 7]

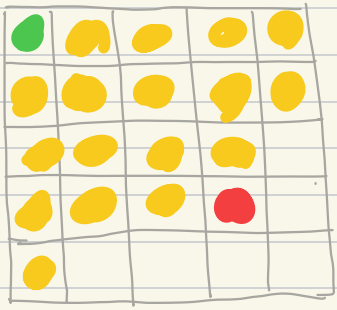


Topological Sort

MATRIX



DFS



BFS

Backtracking

When to use)

- combinatorial problems (combinations, permutations, subsets)
- constraint satisfaction (Sudoku, N-Queens)
- Prune paths using constraints to reduce search space

What is it)

- although closely related to DFS, backtracking will focus more on finding solutions while validating their correctness. If a solution doesn't work, you backtrack by returning to the previous recursion and trying a different option. It also uses constraints to eliminate branches that can't lead to a valid solution.